

Deteccao de Fraudes em Cartoes com Lakehouse Local, Apache Spark e Avaliacao Temporal Reprodutivel

Rafael Luciano -- Junho de 2026

Resumo. A deteccao de fraude em pagamentos combina uma classe positiva rara, custos assimetricos de erro e necessidade de resposta rapida. Este trabalho apresenta e avalia uma arquitetura lakehouse local para o problema, integrando Kafka, Apache Spark Structured Streaming, HDFS, Apache Iceberg, Hive Metastore, Airflow e Spark MLlib. O experimento usa as 284.807 transacoes do conjunto ULB e separa cronologicamente treino, validacao e teste em 60/20/20. Dois ensembles Random Forest--Gradient-Boosted Trees ponderados por classe sao comparados em cinco sementes; a selecao usa somente PR-AUC de validacao e o teste permanece isolado. O candidato selecionado alcançou, no bloco de teste, PR-AUC de 0,749825 0,016599, ROC-AUC de 0,978700 0,004384, precision de 0,861579 0,025262 e recall de 0,735135 0,015408. A materializacao operacional produziu 284.807 scores e 562 alertas. Alem de resultados preditivos, o estudo mostra como uma camada de tabelas transacionais e um catalogo compartilhado sustentam rastreabilidade, reprocessamento e analise de qualidade.

Palavras-chave: deteccao de fraude; lakehouse; Apache Spark; streaming; Apache Iceberg; avaliacao temporal; dados desbalanceados.

Introduction

Pagamentos digitais tornaram a analise antifraude uma tarefa simultaneamente estatistica e operacional. A decisao precisa ocorrer no intervalo entre a chegada do evento e sua autorizacao, mas o resultado verdadeiro da transacao normalmente so e conhecido depois. Alem disso, uma pequena fracao dos registros representa fraude. Uma acuracia elevada, portanto, pode ser obtida por um classificador inutil que chama todos os eventos de legitimos. A consequencia pratica e dupla: falsos negativos expoem a instituicao a perdas; falsos positivos interrompem clientes legitimos e aumentam o custo de revisao humana.

A literatura de fraude enfatiza que a modelagem deve considerar desbalanceamento, mudanca temporal e custos de decisao, em vez de tratar a classificacao binaria como um problema i.i.d. comum [ngai2011,dalpozzolo2015,dalpozzolo2018]. Em particular, um teste cronologicamente posterior e mais proximo do uso operacional do que uma particao aleatoria: ele reduz a possibilidade de registros de um mesmo periodo aparecerem nos dois lados da avaliacao e oferece uma visao mais honesta da generalizacao ao futuro. Essa preocupacao e especialmente relevante quando as variaveis descrevem padroes transacionais, pois comportamento

fraudulento e comportamento legitimo podem evoluir.

Ao lado do modelo, a infraestrutura precisa preservar o caminho do dado. Uma solucao que recebe eventos em streaming, treina em lote e depois produz indicadores analiticos tende a acumular sistemas separados, copias de dados e regras dificeis de auditar. O conceito de lakehouse procura combinar armazenamento aberto de um data lake com controle de metadados, confiabilidade de tabelas e desempenho analitico tradicionalmente associados ao data warehouse [armbrust2021]. Essa convergencia e adequada a fraude: os mesmos dados podem sustentar ingestao, preparacao, treinamento, inferencia, alertas e consultas retrospectivas.

Este artigo apresenta uma implementacao local, deliberadamente reprodutivel, de um lakehouse para deteccao de fraude. A contribuicao principal nao e propor um novo algoritmo, mas estabelecer uma pipeline experimental completa e auditavel com ferramentas abertas. O trabalho: (i) organiza eventos ULB em camadas Bronze, Silver e Gold sobre Iceberg/HDFS; (ii) realiza inferencia apos Silver e antes de Gold, preservando a ordem operacional; (iii) define um protocolo temporal de treino, validacao, teste e cinco repeticoes; e (iv) reporta metricas de classificacao, matrizes de confusao, scores, alertas, qualidade e tempos de execucao. Trabalhos recentes sobre lakehouse para risco de pagamentos reforcam a utilidade de combinar processamento de eventos e analise em lote, embora frequentemente descrevam arquiteturas em nivel conceitual [battapothu2025,carrington2025,adlermann2024]. Aqui, cada resultado apresentado foi materializado no ambiente executado.

Description

Detecção de fraude e avaliação desbalanceada

Em detecção de fraude, a classe positiva é rara e os erros não têm o mesmo significado. Seja TP o número de fraudes corretamente sinalizadas, FP os alertas incorretos, TN as transações legítimas aceitas e FN as fraudes não sinalizadas. A precisão é $TP/(TP+FP)$, o recall é $TP/(TP+FN)$, enquanto o F1 harmoniza os dois critérios. Essas medidas devem ser lidas em conjunto: aumentar a sensibilidade reduzindo o limiar pode elevar fortemente os falsos positivos; tornar o limiar muito alto pode reduzir a fila de revisão, mas deixar fraudes passar.

A curva ROC e seu AUC continuam úteis para resumir ordenação de scores, mas podem parecer excessivamente favoráveis quando negativos dominam a população. A curva precisão-recall (PR) concentra a análise na classe de interesse e é recomendada para conjuntos desbalanceados [davis2006,saito2015]. Por isso, este estudo declara PR-AUC como métrica primária para escolher hiperparâmetros. ROC-AUC, precisão, recall, F1 e a matriz de confusão são reportados como medidas complementares. A acurácia não é utilizada para seleção.

Outra decisão fundamental é a separação dos dados. Validação cruzada estratificada é comum em problemas estáticos, mas pode subestimar o erro quando há dependência temporal. Estudos de fraude recomendam distinguir o período de treinamento do período futuro de avaliação e tornar explícita a estratégia de atualização [dalpozzolo2018]. O protocolo deste trabalho usa três blocos cronológicos e não permite que o teste influencie a escolha de algoritmo, limiar ou pesos.

Lakehouse e tabelas transacionais abertas

O lakehouse emprega armazenamento de arquivos aberto, mas adiciona um catálogo e operações de tabela para oferecer consistência a workloads de engenharia e análise. Iceberg materializa esse papel no experimento: cada camada é uma tabela catalogada pelo Hive Metastore e mantida no HDFS. O catálogo fornece um nome estável para leitura por Spark, enquanto snapshots e metadados permitem evolução e inspeção das tabelas. Dessa forma, a camada Bronze não é apenas um diretório bruto, e Gold não é apenas uma exportação CSV sem linhagem.

As camadas seguem a convenção medallion. Bronze retém evento, tópico, partição, offset, instante de ingestão e identificador técnico. Silver aplica tipos, remove registros inválidos, cria a label e disponibiliza as features do modelo. Gold recebe scores, alertas, KPIs, métricas de qualidade e resultados experimentais. A separação torna visível onde um resultado foi produzido e permite reprocessar Silver ou Gold sem republicar o arquivo de origem. A visão é compatível com arquiteturas multi-tier discutidas para domínios financeiros regulados

[bfsilakehouse2023,allam2024].

Processamento distribuído, eventos e orquestração

Apache Spark é um motor unificado para processamento em memória, SQL, streaming estruturado e aprendizado de máquina [zaharia2016]. No projeto, o mesmo ecossistema executa a limpeza batch, o treinamento MLlib, a leitura Structured Streaming de Kafka, a gravação Iceberg e as agregações Gold. Isso reduz a troca de formatos entre etapas e permite reutilizar o catálogo e as mesmas definições de schema.

Kafka é o barramento de eventos. O produtor publica uma transação ULB por mensagem JSON em transactions.raw; o streaming Bronze a lê a partir do início e persiste offsets no checkpoint HDFS. Kafka foi concebido como sistema de mensagens distribuído para processamento de logs, com retenção e alto throughput [kreps2011]. Structured Streaming oferece o modelo de micro-batches e checkpoint para que a transformação seja expressa como consulta incremental, aproximando processamento de eventos e processamento histórico [akidau2015].

Airflow tem papel de orquestrador: suas DAGs disparam os jobs de inicialização, ingestão, Silver, treinamento, agregação e qualidade. Ele não controla um loop contínuo de consumo Kafka; os streams pertencem ao Spark. Essa separação evita usar um agendador batch como mecanismo de streaming e preserva a responsabilidade de cada componente. Artigos aplicados recentes também relacionam orquestração e camadas analíticas à rastreabilidade em ambientes financeiros [bfsilakehouse2023].

Methodology

Visão geral da arquitetura

A arquitetura foi implementada inteiramente em contêineres Docker Compose. Os serviços são Kafka e ZooKeeper, HDFS NameNode/DataNode, Hive Metastore com PostgreSQL, Spark Master e Worker, Airflow com PostgreSQL e Kafka UI. O armazenamento principal é HDFS; Iceberg usa Hive Catalog no endereço do metastore. As versões são fixadas no repositório: Spark 3.5.1, Iceberg 1.5.2, Kafka Confluent 7.6.1, Hadoop 3.2.1, Hive Metastore 3.1.3 e Airflow 2.9.2.

[Figura 1 -- inserir manualmente] Diagrama da arquitetura: CSV ULB produtor Kafka Bronze/Iceberg-HDFS Silver treinamento MLlib scoring streaming ou batch Gold, Power BI e Airflow.

O desenho atende a dois caminhos. O caminho online recebe e persiste eventos, enquanto o caminho analítico produz modelos, analisa resultados e prepara indicadores. O score é calculado depois da Silver, pois somente ali as features têm schema e tipos controlados; ele ocorre antes da Gold, pois Gold armazena o produto da decisão, e não a etapa de decisão. Essa ordem também permite que alertas e scores sejam publicados no Kafka sem aguardar agregações diárias.

Ingestão e camada Bronze

O arquivo `creditcard.csv` é validado antes da publicação. O produtor verifica a presença de `Time`, `V1` a `V28`, `Amount` e `Class`; cada linha recebe um identificador determinístico e é enviada a Kafka. Na execução, foram usados lotes de 5.000 mensagens. O produtor publicou 284.807 eventos em 164,6 s, distribuídos em três partições do tópico. O checkpoint Bronze confirmou os offsets finais 95.515, 95.267 e 94.025, cuja soma é o número total de eventos.

O job `kafka_to_bronze.py` lê JSON, preserva o offset Kafka e grava em `local.bronze.transactions_raw`. A combinação de `transaction_id` e offset cria `raw_event_id`, permitindo distinguir replay técnico de identidade de negócio. O checkpoint fica no HDFS; assim, uma reinicialização do stream continua a partir do ponto processado. Essa etapa foi mantida mesmo para o conjunto histórico porque o objetivo experimental inclui validar a rota de ingestão que seria usada para dados em produção.

Silver: qualidade, schema e features

Silver é gerada pelo job `bronze_to_silver.py`. Todas as variáveis do dataset são convertidas explicitamente para `double`; a classe é disponibilizada como `label`; valores nulos das features são preenchidos com zero e labels fora de `\0,1\` são excluídos. A tabela `local.silver.transactions_clean` preserva metadados técnicos e `local.silver.transactions_features` inclui um vetor de features para consumo analítico.

O desbalanceamento é tratado por pesos de classe no treinamento. Para cada classe c , o peso empregado é $w_c = N / (2N_c)$, em que N é o número de registros de treinamento e N_c o número de elementos da classe. Os pesos são calculados exclusivamente no bloco de treino em cada etapa de ajuste, não no conjunto completo. Essa estratégia preserva o conjunto original e evita introduzir síntese externa, alinhando-se a uma comparação controlada com algoritmos Spark MLlib.

Protocolo experimental e prevenção de vazamento

O protocolo foi registrado em `configs/app/experiment.yaml`. A variável `Time` determina dois pontos de corte aproximados por quantil: 120.390 s e 145.229 s. O primeiro bloco contém 170.229 transações (60%), o segundo 56.732 (20%) e o bloco de teste 56.765 (20%). Os tamanhos diferem levemente das proporções nominais porque existem eventos com o mesmo instante; a regra de fronteira preserva o ordenamento. Os blocos contêm, respectivamente, 323, 95 e 74 fraudes.

O experimento adota cinco sementes: 42, 52, 62, 72 e 82. Não existe um único número universal de repetições que seja "gold standard" para fraude; a literatura salienta que a avaliação deve refletir a dependência temporal e a variabilidade do método [dalpozzolo2018, hand2009]. Cinco repetições são usadas aqui como compromisso usual de estabilidade para algoritmos estocásticos em ambiente local: permitem reportar média e desvio-padrão, sem confundir o teste com uma busca exaustiva de configurações. O conjunto de teste permanece fechado durante as dez execuções de validação.

Dois candidatos foram definidos antes da execução. O candidato compact usa Random Forest com 80 árvores e profundidade máxima 8, mais GBT com 50 iterações e profundidade 3. O candidato extended usa 120 árvores/profundidade 10 e GBT de 60 iterações/profundidade 4. A grade é deliberadamente pequena: árvores profundas e muitas iterações aumentariam custo e risco de ajuste excessivo para uma demonstração arquitetural, enquanto duas capacidades permitem observar se maior complexidade melhora a ordenação PR no período de validação. Cada candidato é treinado cinco vezes sobre o mesmo bloco de treino.

Para cada ajuste, o score do ensemble é a média simples das probabilidades de fraude de Random Forest e GBT: $s(x) = p_{RF}(y=1|x) + p_{GBT}(y=1|x)/2$. O

limiar e buscado na grade 0,05, 0,10, , 0,95. A política seleciona o maior recall com precisão mínima de 0,70; se nenhum limiar satisfizer o piso, usa o maior F1. A decisão de configuração usa a média de PR-AUC na validação, com F1 como desempate. Apenas após a escolha são carregados os cinco modelos vencedores e calculadas as métricas no teste. Por fim, um modelo de implantação é ajustado sobre treino mais validação; seu limiar é a mediana dos cinco limiares de validação. Esse último modelo materializa Gold, mas suas métricas retrospectivas não substituem o teste isolado.

Treinamento, registro e inferência

O treinamento usa exclusivamente RandomForestClassifier e GBTClassifier do Spark MLlib, ambos com weightCol. Cada modelo de validação é salvo no HDFS em um caminho que contém identificador do experimento, candidato e semente. As tabelas local.gold.experiment_metrics e local.gold.experiment_summary registram seed, split, limiar, matriz de confusão, PR-AUC, ROC-AUC, média e desvio-padrão. O arquivo local/models/latest_metadata.json registra versão, colunas, pesos, pontos de corte e caminhos dos modelos.

Para demonstrar materialização reproduzível, o job score_silver_batch.py aplica o modelo de implantação a toda a Silver. O job de streaming existente aplica a mesma lógica em micro-batches para novos registros. Em ambos os casos, fraud_score, probabilidades individuais, limiar, versão do modelo e instante de score são escritos em local.gold.fraud_scores; previsões positivas também entram em local.gold.fraud_alerts. A rotina Gold gera KPIs horários e diários e exporta CSV para Power BI. A rotina de qualidade mede contagem, nulidade, distribuições e outros indicadores, gravando 151 métricas na tabela Gold.

Ambiente e verificação

O experimento foi executado em 21 de junho de 2026 no ambiente local Docker do projeto, com um Spark Worker configurado com 2 cores e 3 GB de memória. A execução completa incluiu reinicialização dos volumes para evitar mistura com a amostra exploratória anterior, inicialização Iceberg, criação dos tópicos, ingestão, Bronze, Silver, experimento, scoring batch, Gold e qualidade. Os testes unitários do repositório foram executados no container Spark e obtiveram 12 testes aprovados.

Os tempos observados foram 164,6 s para publicação Kafka, 66,7 s para Bronze--Silver, 1.224,8 s para o experimento completo de modelos, 74,0 s para scoring batch, 44,7 s para agregações Gold e 68,2 s para qualidade. Esses tempos não são benchmark de escala, pois incluem um único worker e downloads iniciais de dependências no primeiro job. Eles documentam, contudo, que a pipeline foi executada de ponta a ponta e oferecem uma base para comparação de futuras configurações.

Results

Dataset ULB e integridade da execucao

O conjunto Credit Card Fraud Detection foi disponibilizado pelo Machine Learning Group da Universite Libre de Bruxelles em parceria com Worldline [ulb2013]. Ele contem transacoes de cartoes europeus realizadas durante dois dias de setembro de 2013. Das 284.807 instancias, 492 sao fraudulentas, o que corresponde a aproximadamente 0,1727%. Por razoes de confidencialidade, as componentes V1--V28 resultam de transformacao PCA; Time representa segundos desde a primeira transacao e Amount e o valor monetario. A label Class e usada apenas para treinamento e avaliacao offline; em uma implantacao real, o rotulo chegaria com atraso.

A Tabela tab:layers confirma a integridade basica do fluxo. Bronze, Silver e Gold de scores preservaram as 284.807 transacoes. Isso mostra que o produtor, as tres particoes Kafka, o stream Iceberg e o job Silver nao perderam ou duplicaram registros no experimento reinicializado. A tabela de alertas contem apenas a parcela classificada como suspeita; qualidade registrou 151 medidas auditaveis.

[ht] Tabela/Figura: Contagens materializadas por camada. Ir Artefato & Registros Bronze transactions_raw & 284.807 Silver transactions_clean & 284.807 Gold fraud_scores & 284.807 Gold fraud_alerts & 562 Metricas de qualidade & 151

Selecao por validacao

A Tabela tab:validation mostra as medias das cinco sementes. O candidato compact foi escolhido porque sua PR-AUC media na validacao foi 0,656087, ligeiramente superior a 0,649046 do extended. A diferenca e pequena e os desvios-padrao se sobrepoem; portanto, o resultado nao deve ser lido como evidencia de superioridade ampla da configuracao menor. Ele mostra, no entanto, que aumentar arvores, profundidade e iteracoes nao trouxe ganho consistente no criterio previamente declarado. O extended obteve precision e F1 medias superiores nesse bloco, mas a regra de selecao manteve PR-AUC como objetivo principal para evitar trocar o criterio apos ver os resultados.

[t] Tabela/Figura: Validacao temporal: media desvio-padrao em cinco sementes. O vencedor foi definido por PR-AUC. Irrrrr Candidato & PR-AUC & ROC-AUC & Precision & Recall & F1 & Limiar Compact & 0,656087,030716 & 0,972292,002404 & 0,797130,049850 & 0,736842,021487 & 0,765472,032498 & 0,700,00 Extended & 0,649046,045177 & 0,959786,008407 & 0,860991,063378 & 0,715789,031383 & 0,780548,032860 & 0,580,027386

O limiar 0,70 foi selecionado em todas as cinco execucoes do candidato compacto. Isso e coerente com a politica de precision minima 0,70: no periodo de

validacao, o modelo pode elevar recall ate o ponto em que a fila de alertas ainda preserva qualidade. O limiar do modelo de implantacao foi a mediana desses valores, tambem 0,70. A estabilidade do limiar nao elimina a necessidade de recalibracao futura, pois o dataset e historico e nao representa deriva de conceito em longo prazo.

[Figura 2 -- inserir manualmente] Curvas precision--recall das cinco sementes na validacao e no teste, com destaque para o candidato compacto selecionado.

Teste cronológico isolado

O bloco de teste contém 56.765 transações, das quais 74 são fraudes. A Tabela `tab:test` apresenta cada repetição do candidato selecionado. A média de PR-AUC foi 0,749825 com desvio-padrão 0,016599; ROC-AUC foi 0,978700 com desvio-padrão 0,004384. Precisão média de 0,861579 significa que a maior parte dos alertas do teste correspondeu a fraude, enquanto recall médio de 0,735135 indica que cerca de três quartos das 74 fraudes foram sinalizadas no limiar escolhido. O F1 médio foi 0,793038.

[t] Tabela/Figura: Resultados por semente no bloco temporal de teste. rrrrrrrrrr Run & Seed & Precision & Recall & F1 & PR-AUC & ROC-AUC & TP & FP & TN & FN 1 & 42 & 0,861538 & 0,756757 & 0,805755 & 0,749979 & 0,974861 & 56 & 9 & 56.682 & 18 2 & 52 & 0,843750 & 0,729730 & 0,782609 & 0,756951 & 0,979404 & 54 & 10 & 56.681 & 20 3 & 62 & 0,898305 & 0,716216 & 0,796992 & 0,721545 & 0,973738 & 53 & 6 & 56.685 & 21 4 & 72 & 0,870968 & 0,729730 & 0,794118 & 0,764267 & 0,981293 & 54 & 8 & 56.683 & 20 5 & 82 & 0,833333 & 0,743243 & 0,785714 & 0,756382 & 0,984204 & 55 & 11 & 56.680 & 19 Media & -- & 0,861579 & 0,735135 & 0,793038 & 0,749825 & 0,978700 & -- & -- & -- & -- DP & -- & 0,025262 & 0,015408 & 0,009232 & 0,016599 & 0,004384 & -- & -- & -- & --

Os resultados mostram baixa variação em recall e F1, e variação moderada em PR-AUC. Essa variação é esperada porque há apenas 74 positivos no teste e porque Random Forest e GBT possuem componentes estocásticos. O ponto mais baixo de PR-AUC ocorreu na semente 62, embora sua precisão tenha sido a maior; novamente, uma única métrica não descreve todos os trade-offs. A avaliação por repetição é mais informativa que reportar somente o melhor run, prática que poderia superestimar a capacidade do método.

O teste também evidencia um ponto de interpretação: PR-AUC no teste foi maior que na validação. Isso não deve ser tratado como melhoria causada pelo teste, mas como diferença de dificuldade entre janelas temporais e tamanho pequeno da classe positiva. Como o teste não participou da escolha de configuração, ele continua válido para a estimativa deste protocolo. A diferença reforça a importância de executar reavaliações temporais sucessivas em cenários reais.

Scores e alertas do modelo de implantação

Depois de concluída a avaliação, o modelo compacto de implantação foi treinado com treino e validação e aplicado retrospectivamente a toda a Silver. Esta etapa não é uma nova medida de generalização, porque inclui registros vistos no treino; seu objetivo é demonstrar o produto operacional da pipeline. Foram persistidos 284.807 scores e 562 alertas a limiar 0,70. Dos alertas, 435 eram fraudes reais e 127 eram falsos positivos; 57 fraudes não foram alertadas. A precisão retrospectiva foi $435/562=0,7731$ e o recall retrospectivo foi $435/492=0,8841$.

O recall operacional maior que o recall de teste é esperado porque o modelo de implantação usa 80% dos dados para treinamento e é aplicado também sobre esse período. Por essa razão, a Tabela tab:test, e não estes números retrospectivos, é a referência científica para desempenho preditivo. Os scores Gold, contudo, são valiosos para auditoria, priorização e BI: cada linha preserva probabilidades de RF e GBT, score final, predição, limiar, versão do modelo e timestamp de scoring.

[ht] Tabela/Figura: Materialização operacional sobre toda a Silver. Não é avaliação independente. Ir Medida & Valor Scores produzidos & 284.807 Fraudes rotuladas & 492 Alertas & 562 Verdadeiros positivos & 435 Falsos positivos & 127 Falsos negativos & 57 Precisão retrospectiva & 0,7731 Recall retrospectivo & 0,8841

[Figura 3 -- inserir manualmente] Distribuição de fraud_score e linha vertical no limiar 0,70; opcionalmente sobrepor fraudes reais e alertas.

Desempenho da pipeline e qualidade

A execucao confirma que a arquitetura suporta o fluxo completo no ambiente local. A publicacao Kafka levou 164,6 s e o experimento, que inclui dez ajustes de validacao mais o modelo de implantacao, levou 1.224,8 s (aproximadamente 20,4 min). O scoring batch final levou 74,0 s. Esses numeros sao observacoes do ambiente de teste, e nao afirmacoes de throughput universal: ha apenas um worker, dois cores e 3 GB de memoria. Ainda assim, o fato de todo o conjunto atravessar Kafka, Iceberg e Spark sem perda fornece evidencia de integracao funcional.

As 151 metricas de qualidade incluem contagens, completude e distribuicoes por tabela. A igualdade das contagens entre Bronze, Silver e Gold scores e uma verificacao simples e forte para esta execucao. Em producao, esse conjunto deveria ser complementado por alertas de deriva de distribuicao, atraso de label, mudanca de precision estimada e SLA por micro-batch. A estrutura Iceberg permite que essas verificacoes sejam escritas como tabelas, em vez de logs efemeris.

Conclusions

Este trabalho desenvolveu e executou uma arquitetura lakehouse local para detecção de fraude em cartões. A pipeline integrou Kafka para eventos, Spark Structured Streaming para Bronze e inferência, HDFS e Iceberg para persistência de camadas, Hive Metastore para catálogo, Airflow para orquestração e Spark MLlib para treinamento. O experimento ULB percorreu todas as 284.807 transações, gerou 284.807 scores, 562 alertas, KPIs, exportações CSV e 151 métricas de qualidade.

Do ponto de vista metodológico, a principal decisão foi substituir uma divisão aleatória simples por protocolo temporal 60/20/20, com validação separada, teste isolado e cinco sementes. O candidato compacto foi selecionado por PR-AUC de validação e obteve PR-AUC médio de 0,749825 e precisão média de 0,861579 no teste. Os valores mostram que o modelo identifica uma parcela relevante das fraudes mantendo uma fila de alertas relativamente precisa, mas também deixam claro que ainda há falsos negativos e que a escolha do limiar depende do custo real de revisão e perda.

Há limitações importantes. O ULB é histórico, tem features anonimizadas por PCA e representa somente dois dias; logo, não demonstra deriva de conceito, disponibilidade tardia de labels ou comportamento de clientes em longo prazo. O experimento utiliza um único worker local e não mede escalabilidade horizontal, latência ponta a ponta por evento ou disponibilidade sob falha. A grade de hiperparâmetros foi pequena, os modelos não incluem variáveis de grafo ou contexto de entidade e o ensemble por média não substitui uma análise de custo monetário. Também não foi realizado teste estatístico de diferença entre configurações, pois os cinco runs compartilham os mesmos blocos temporais.

Como trabalhos futuros, propõe-se avaliar janelas temporais deslizantes e prequential, incorporar custos por transação e limites de capacidade de revisão, comparar diferentes políticas de recalibração, medir throughput com múltiplos workers e adicionar monitoramento de deriva. A extensão com atributos de conta, comerciante e dispositivo permitiria investigar modelos relacionais, desde que preservada a separação entre avaliação e dados futuros. O estudo pode ser reproduzido pelo código no GitHub em https://github.com/rafaellucian0/fraud_detection-big_data, que inclui Docker Compose, configurações, jobs Spark, DAGs Airflow, testes e instruções para posicionar o dataset ULB.

References

99

- [akidau2015] Akidau, T.; Chernyak, S.; Lax, R.; et al. (2015). The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment*, 8(12), 1792--1803.
- [adlermann2024] Adlermann, J. F. (2024). A GRA-Enhanced Cloud AI Framework for Petabyte-Scale Multi-Tenant Environments: Multivariate Classification for Credit Card Fraud Detection and Adaptive Risk Analytics. *International Journal of Engineering & Extended Technologies Research*, 6(6).
- [allam2024] Allam, K. (2024). Cloud-Based Data Hubs and SQL Pipelines for Real-Time Financial Analytics. *International Journal of Emerging Trends in Computer Science and Information Technology*, 5(4), 94--104.
- [armbrust2021] Armbrust, M.; Ghodsi, A.; Xin, R.; Zaharia, M. (2021). Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. *Proceedings of CIDR 2021*.
- [battapothu2025] Battapothu, V. (2025). Lakehouse-Powered Payment Risk at Scale. *Manuscripto tecnico independente*.
- [bfsilakehouse2023] Oliveira, M.; Iyer, R. (2023). Migrating BFSI Data Workloads to Cloud-Native Environments: A Case Study on Multi-Tier Data Lakehouse Architectures with AWS Redshift, Athena, and Intelligent Orchestration for Compliance. *Journal of Engineering, Mechanics and Modern Architecture*, 2(11), 50--61.
- [carrington2025] Carrington, N. L. (2025). AI-Enabled Cloud Lakehouse for Large-Scale Data Warehousing: SAP Integration for Secure Analytics and Tableau-Driven Reporting. *International Journal of Research and Applied Innovations*, 8(5).
- [dalpozzolo2015] Dal Pozzolo, A.; Caelen, O.; Le Borgne, Y.-A.; Waterschoot, S.; Bontempi, G. (2015). Calibrating Probability with Undersampling for Unbalanced Classification. *IEEE Symposium Series on Computational Intelligence*, 159--166.
- [dalpozzolo2018] Dal Pozzolo, A.; Caelen, O.; Johnson, R. A.; Bontempi, G. (2018). Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy. *IEEE Transactions on Neural Networks and Learning Systems*, 29(8), 3784--3797.
- [davis2006] Davis, J.; Goadrich, M. (2006). The Relationship Between Precision-Recall and ROC Curves. *Proceedings of the 23rd International Conference on Machine Learning*, 233--240.
- [hand2009] Hand, D. J. (2009). Measuring Classifier Performance: A Coherent Alternative to the Area Under the ROC Curve. *Machine Learning*, 77, 103--123.
- [kreps2011] Kreps, J.; Narkhede, N.; Rao, J. (2011). Kafka: A Distributed Messaging System for Log Processing. *Proceedings of NetDB*, 1--7.
- [ngai2011] Ngai, E. W. T.; Hu, Y.; Wong, Y. H.; Chen, Y.; Sun, X. (2011). The Application of Data Mining Techniques in Financial Fraud Detection: A Classification Framework and an Academic Review of Literature. *Decision Support Systems*, 50(3), 559--569.
- [saito2015] Saito, T.; Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE*, 10(3), e0118432.
- [ulb2013] Worldline and Machine Learning Group, Universite Libre de Bruxelles. (2013). Credit Card Fraud Detection Dataset. Kaggle Dataset: mlg-ulb/creditcardfraud.
- [zaharia2016] Zaharia, M.; Xin, R. S.; Wendell, P.; et al. (2016). Apache Spark: A Unified Engine for Big Data Processing. *Communications of the ACM*, 59(11), 56--65.